

МИНОБРНАУКИ РОССИИ
ФГБОУ ВО «ИжГТУ имени М.Т. Калашникова»
Факультет «Информационные технологии»
Кафедра «Программное обеспечение»

КУРСОВАЯ РАБОТА
по дисциплине:
«Алгоритмы и структуры данных»
на тему: «Сравнительный анализ алгоритмов сортировки: сортировка выбором (Selection Sort)
и пирамидальная сортировка (Heap Sort)»

Выполнил:

студент гр. Б24-191-3

Т.И. Ишматов

Проверил:

д.э.н, доцент, заведующий кафедрой

М.В. Леонов

Ижевск, 2025

Оглавление

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ	3
ССЫЛКА НА КОД В ПУБЛИЧНОМ РЕПОЗИТОРИИ	4
ВВЕДЕНИЕ	5
ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	6
1.1 Сортировка выбором	6
1.1.1 Работа алгоритма сортировки выбором на примере	6
1.1.2 Псевдокод алгоритма сортировки выбором	7
1.1.3 Оценка сложности алгоритма сортировки выбором	7
1.2 Пирамидальная сортировка	8
1.2.1 Работа алгоритма пирамидальной сортировки на примере	9
1.2.2 Псевдокод алгоритма пирамидальной сортировки	10
1.2.3 Оценка сложности алгоритма пирамидальной сортировки	11
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ	12
2.2 Реализация алгоритмов	12
2.2.1 реализация сортировки выбором	12
2.2.2 реализация пирамидальной сортировки	13
2.3 Результаты экспериментального тестирования	14
ЗАКЛЮЧЕНИЕ	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Тема курсовой работы: «Сравнительный анализ алгоритмов сортировки: сортировка выбором (Selection Sort) и пирамидальная сортировка (Heap Sort)».

Цель курсовой работы: проведение сравнительного анализа выбранных алгоритмов, включая теоретическое исследование их вычислительной сложности, практическую реализацию и экспериментальное тестирование производительности.

Задачи курсовой работы:

- исследовать теоретические основы выбранных алгоритмов: изучить принципы работы каждого алгоритма, проанализировать вычислительную сложность в лучшем, среднем и худшем случаях, выявить достоинства и недостатки каждого алгоритма;
- реализовать алгоритмы на выбранном языке программирования: обосновать выбор языка программирования, разработать программную реализацию алгоритмов, обеспечить корректность работы реализаций;
- провести экспериментальное исследование эффективности алгоритмов: разработать методику тестирования алгоритмов, сгенерировать тестовые данные различных типов и объемов, выполнить замеры времени выполнения алгоритмов;
- проанализировать полученные результаты: сравнить экспериментальные данные с теоретическими оценками, определить оптимальные области применения каждого алгоритма, сформулировать практические рекомендации по использованию.

ССЫЛКА НА КОД В ПУБЛИЧНОМ РЕПОЗИТОРИИ

Публичный репозиторий: GitHub

Ссылка: <https://github.com/timofey18269/kyrsovie>

ВВЕДЕНИЕ

Исследования алгоритмов сортировки всегда являлись актуальной темой, так как во всех сферах связанных с работой с большими объёмами данных необходимо их упорядочивать. Следовательно чем лучше работает алгоритм сортирующий данные, тем больше времени и ресурсов получится сэкономить, что в свою очередь повышает эффективность всего процесса.

Существует множество классических алгоритмов сортировки. Некоторые из наиболее известных среди них:

1. Сортировка пузырьком (Bubble Sort)
2. Сортировка вставками (Insertion Sort)
3. Сортировка выбором (Selection Sort)
4. Быстрая сортировка (Quicksort)
5. Пирамидальная сортировка (сортировка кучей) (Heapsort)

Для сравнительного анализа в данной работе выбраны алгоритм сортировки выбором и пирамидальная сортировка. Их сравнение наглядно демонстрирует эволюционный скачок от тривиальных, учебных алгоритмов к эффективным, пригодным для промышленного применения. Анализ позволяет оценить, как замена наивной стратегии поиска минимума (линейный проход в сортировке выбором) на оптимизированную структуру данных (куча, обеспечивающая логарифмическое время извлечения максимума) значительно улучшает производительность.

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Сортировка выбором

Сортировка выбором — это алгоритм, который делит массив на две части: отсортированную и неотсортированную. На каждом шаге алгоритм находит минимальный (или максимальный) элемент в неотсортированной части и меняет его местами с первым элементом неотсортированной части. Этот процесс повторяется, пока весь массив не будет отсортирован.

Как работает алгоритм (в случае с поиском минимальных элементов):

1. Разделение массива: Массив условно делится на две части — уже отсортированную, которая изначально пуста, и неотсортированную.
2. Поиск минимума: Алгоритм ищет минимальный элемент в неотсортированной части массива.
3. Обмен: Найденный минимальный элемент меняется местами с первым элементом неотсортированной части.
4. Сдвиг границы: Граница между отсортированной и неотсортированной частями смещается на один элемент вправо, так как первый элемент неотсортированной части теперь находится на своём отсортированном месте.
5. Повторение: Шаги 2–4 повторяются до тех пор, пока вся неотсортированная часть массива не станет пустой.

1.1.1 Работа алгоритма сортировки выбором на примере

Массив	Описание шага
<i>Первый проход (текущий массив начинается с первого элемента)</i>	
5 4 1 2 3	Находим первый минимальный элемент — 1
1 4 5 2 3	Меняем минимальный и первый элементы местами
<i>Второй проход (текущий массив начинается со второго элемента)</i>	
1 4 5 2 3	Находим следующий минимальный элемент — 2
1 2 5 4 3	Меняем минимальный и второй элементы местами
<i>Третий проход (текущий массив начинается с третьего элемента)</i>	
1 2 5 4 3	Находим следующий минимальный элемент — 3
1 2 3 4 5	Меняем минимальный и третий элементы местами
<i>Четвертый проход (текущий массив начинается с четвёртого элемента)</i>	
1 2 3 4 5	Находим следующий минимальный элемент — 4 .
1 2 3 4 5	Меняем четвёртый элемент местами с самим собой. Массив отсортирован.

1.1.2 Псевдокод алгоритма сортировки выбором

```
function selectionSort(arr):  
  
    n = length(arr)  
    // определяем длину массива  
  
    for i from 0 to n-1:  
  
        minIndex = i  
        // обозначаем границу неотсортированной части  
  
        for j from i+1 to n-1:  
            if arr[j] < arr[minIndex]:  
                minIndex = j  
        // ищем минимальный элемент в неотсортированной части  
  
        swap(arr[i], arr[minIndex])  
        // ставим минимальный элемент в начало неотсортированной части  
  
    return arr
```

1.1.3 Оценка сложности алгоритма сортировки выбором

Алгоритм всегда работает за $O(n^2)$, так как всегда выполняет одинаковое количество сравнений. На каждой итерации внешнего цикла внутренний цикл выполняет сравнения для всех оставшихся элементов(на каждой следующей итерации нужно выполнить на 1 сравнение меньше, чем на предыдущей).

Количество сравнений: $n + (n-1) + (n-2) + \dots + 1 = n*n/2 \approx O(n^2)$

1.2 Пирамидальная сортировка

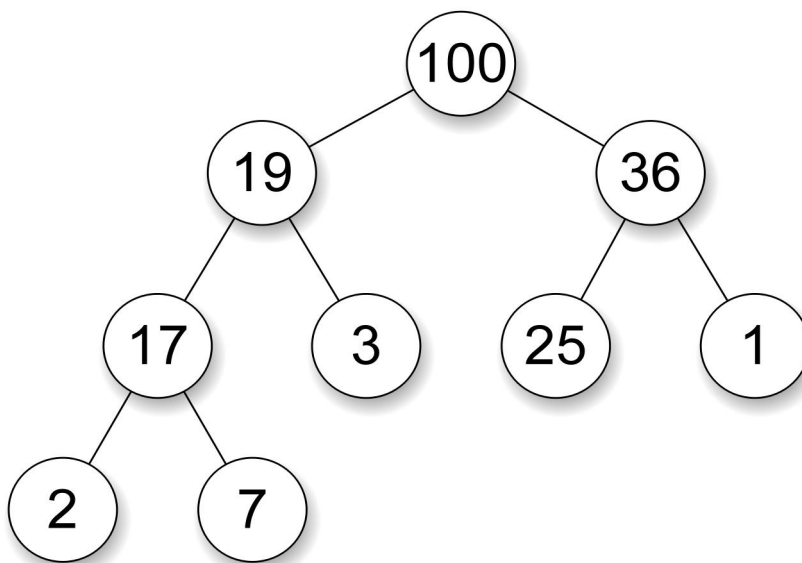
Сортировка кучей, пирамидальная сортировка (Heap sort) — алгоритм сортировки, использующий структуру данных двоичная куча.

Двоичная куча, пирамида, или сортирующее дерево — такое двоичное дерево, для которого выполнены три условия:

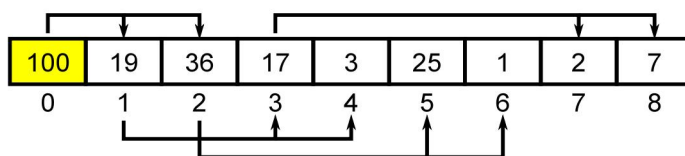
1. Значение в любой вершине не меньше, чем значения её потомков.
2. Глубина всех листьев (расстояние до корня) различается не более чем на 1 слой.
3. Последний слой заполняется слева направо без «дырок».

Удобная структура данных для сортирующего дерева — массив A , у которого первый элемент, $A[0]$ — элемент в корне, а потомками элемента $A[i]$ являются $A[2i+1]$ и $A[2i+2]$ (при нумерации элементов с нулевого).

Tree representation



Array representation



Алгоритм сортировки будет состоять из двух основных шагов:

1. Выстраиваем элементы массива в виде сортирующего дерева
2. Будем удалять элементы из корня по одному за раз и перестраивать дерево.

То есть :

- меняем местами $a[0]$ и $a[n-1]$
- преобразовываем $a[0], a[1], \dots, a[n-2]$ в сортирующее дерево
- меняем местами $a[0]$ и $a[n-2]$
- преобразовываем $a[0], a[1], \dots, a[n-3]$ в сортирующее дерево

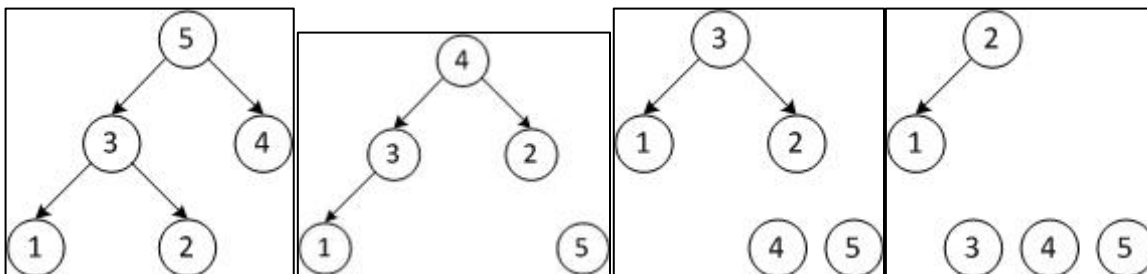
Процесс продолжается до тех пор, пока в сортирующем дереве не останется один элемент.

Тогда $a[0], a[1], \dots, a[n-1]$ - упорядоченная последовательность.

1.2.1 Работа алгоритма пирамидальной сортировки на примере

Массив	Описание шага
5 3 4 1 2	Строим кучу из исходного массива
<i>Первый проход</i>	
2 3 4 1 5	Меняем местами первый и последний элементы
4 3 2 1 5	Строим кучу из первых четырёх элементов
<i>Второй проход</i>	
1 3 2 4 5	Меняем местами первый и четвёртый элементы
3 1 2 4 5	Строим кучу из первых трёх элементов
<i>Третий проход</i>	
2 1 3 4 5	Меняем местами первый и третий элементы
2 1 3 4 5	Строим кучу из двух элементов
<i>Четвёртый проход</i>	
1 2 3 4 5	Меняем местами первый и второй элементы
1 2 3 4 5	Массив отсортирован

* красным цветом обозначена отсортированная часть массива



Двоичная куча

Первый проход

Второй проход

Третий проход



Результат

1.2.2 Псевдокод алгоритма пирамидальной сортировки

```
function heapify(array, n, i):  
    largest = i  
    left = 2*i + 1  
    right = 2*i + 2  
  
    // выбираем наибольший из конкретного элемента двоичной кучи и двух  
    // его потомков и продвигаем (наибольший) его вверх  
    if left < n and array[left] > array[largest]:  
        largest = left  
    if right < n and array[right] > array[largest]:  
        largest = right  
    if largest != i:  
        swap(array[i], array[largest])  
        heapify(array, n, largest)  
  
function heapSort(array):  
    // определяем длину массива  
    n = length(array)  
  
    // функция heapify применённая к первым n/2 элементам массива строит  
    // из неупорядоченного массива двоичную кучу  
    for i = n/2 - 1 down to 0:  
        heapify(array, n, i)  
  
    for i = n - 1 down to 0:  
        // убираем наибольший элемент двоичной кучи в конец массива  
        // и уменьшаем размер двоичной кучи на один.  
        swap(array[0], array[i])  
  
        // функция heapify применённая к первому элементу массива  
        // восстанавливает двоичную кучу после того как, первый и  
        // последний элементы меняются местами  
        heapify(array, i, 0)
```

1.2.3 Оценка сложности алгоритма пирамидальной сортировки

Чтобы построить двоичную кучу из не упорядоченного массива, нужно $n/2$ раз применить функцию «heapify», которая выполнится в лучшем случае за $O(1)$, в случае если массив уже представляет собой двоичную кучу, а в худшем за $O(\log n)$

Функция «heapify» выполняется максимум за $O(\log n)$.

Далее чтобы отсортировать массив нужно n раз вызвать функцию «heapify», которая всегда будет выполняться за $O(\log (n - i)) \approx O(\log n)$, так как мы всегда всегда меняем наибольший элемент текущей двоичной кучи с одним из элементов не имеющих потомков, то есть чтобы восстановить двоичную кучу, придётся опустить элемент с верхнего уровня кучи на нижний, при этом сама двоичная куча после каждой итерации будет становиться на 1 меньше.

Таким образом :

В худшем случае: $n/2 * \log_2 n + n * \log_2 n \approx 1.5 * n * \log_2 n$

В лучшем случае: $n/2 + n * \log_2 n$

Разница между худшим и лучшим случаями достаточно не большая, поэтому в среднем время выполнения алгоритма принимается как константное и оно равно: $O(n * \log_2 n)$

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Экспериментальный стенд и инструменты

Используемый язык программирования: c++

версия языка: c++23

Выбор языка обоснован тем, что C++ высокопроизводительный и имеет все необходимые функции для реализации исследуемых алгоритмов.

процессор: intel core i5-12400

оперативная память: 32гб ddr5 4800 мгц

материнская плата: GIGABYTE B760 DS3H

операционная система: windows 11 pro

2.2 Реализация алгоритмов

2.2.1 реализация сортировки выбором

```
void selectionSort(int a[], int n) {
    int cur_min_indx;
    for (int i = 0; i < n - 1; i++) {
        cur_min_indx = i;
        for (int j = i+1; j < n; j++) {
            if (a[j] < a[cur_min_indx]) {
                cur_min_indx = j;
            }
        }
        swap(a[cur_min_indx], a[i]);
    }
}

int main()
{
    // n - количество элементов в массиве
    int* a = new int[n];
    // заполнение массива тестовыми данными
    selectionSort(a, n);
}
```

2.2.2 реализация пирамидальной сортировки

```
void heapify(int arr[], int k, int i)
{
    previous_largest = -1, largest = i, l, r;

    while (previous_largest != largest) {
        previous_largest = largest;
        l = 2 * largest + 1;
        r = 2 * largest + 2;

        if (l < k && arr[l] > arr[largest])
            largest = l;

        if (r < k && arr[r] > arr[largest])
            largest = r;

        if (previous_largest != largest)
        {
            swap(arr[previous_largest], arr[largest]);
        }
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i >= 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

// функция heapify здесь реализована не в виде рекурсии, а в виде цикла. В данном случае дополнительных затрат по памяти не требуется и при этом на больших объёмах данных это увеличит скорость работы программы.

2.3 Результаты экспериментального тестирования

Данные для тестирования генерируются случайным образом с помощью библиотеки «random» для языка «с++» и сохраняются в текстовых файлах (отдельный файл для каждого размера набора данных)

Размеры наборов тестовых данных: 10,20,50,100,200,500,1000,2000,5000,10000 элементов.

Количество наборов для каждой размерности: 500

Тестовые данные - случайные числа в диапазоне: [-2147483647, 2147483647]

Для замера времени используется библиотека «chrono»

Функция для замера времени: `chrono::high_resolution_clock::now()`

Таблица 1. Среднее время выполнения алгоритмов в секундах

Размер набора данных	Сортировка выбором	Пирамидальная сортировка	Встроенная функция сортировки языка с++ (sort)
10	0.0000005	0.0000006	0.0000007
20	0.0000010	0.0000012	0.0000016
50	0.0000035	0.0000033	0.0000052
100	0.0000098	0.0000072	0.0000119
200	0.0000288	0.0000160	0.0000276
500	0.0001350	0.0000482	0.0000780
1000	0.0004761	0.0001059	0.0001744
2000	0.0017576	0.0002356	0.0003906
5000	0.0104336	0.0006653	0.0010563
10000	0.0405076	0.0013914	0.0022464

График 1

Пирамидальная сортировка (фиолетовым)

Встроенная функция сортировки языка c++ (sort) (синим)

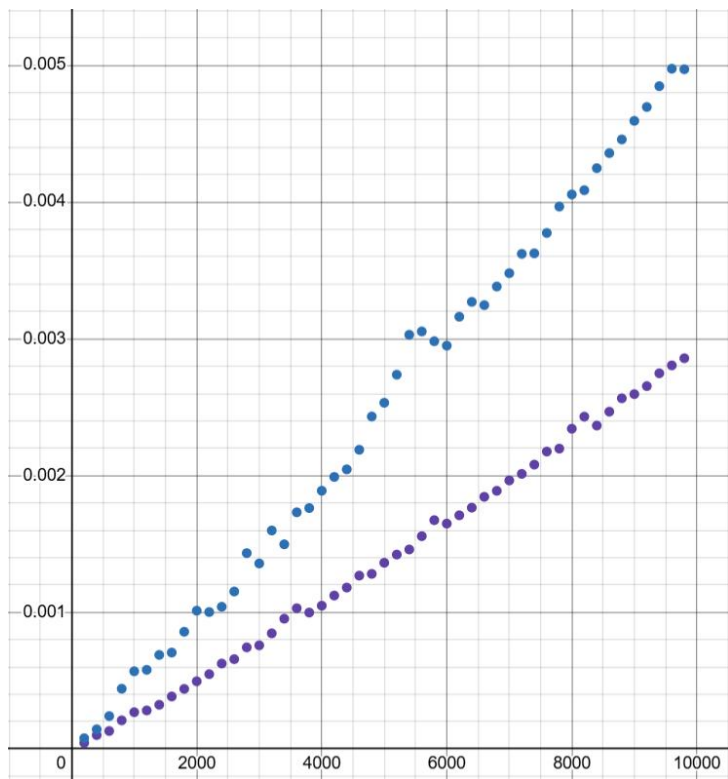
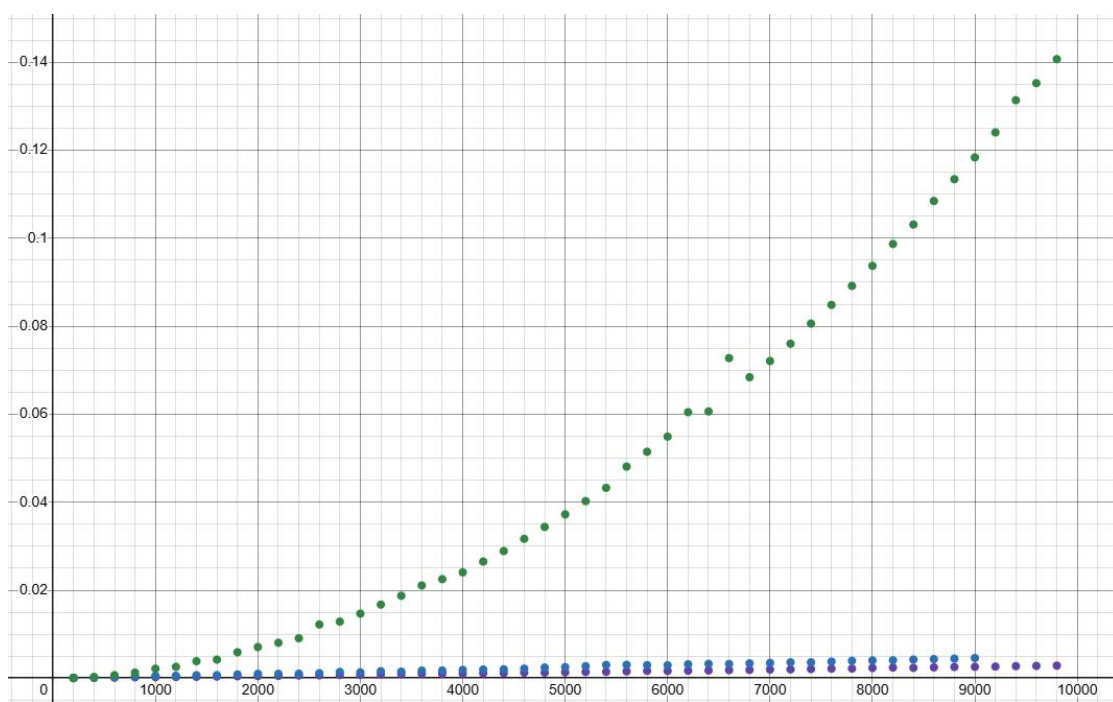


График 2

Сортировка выбором (зелёным)

Пирамидальная сортировка (фиолетовым)

Встроенная функция сортировки языка c++ (sort) (синим)



Графики для наглядности построены отдельно, так как требуют разного масштаба, и меньшим разрывом по количеству данных в наборе. По графику сортировки выбором видно что он соответствует квадратичной сложности, а графики пирамидальной сортировки и встроенной функции сортировки языка c++ (sort) соответствует графику « $y = x * \log_2(x)$ » на большом масштабе.

Для анализа рассмотрим набор данных размером 10000, 1000:

Сравнение теоретической сложности:

$$\frac{10000^2 / 2}{10000 * \log_2(10000)} = 376$$

$$\frac{1000^2 / 2}{1000 * \log_2(1000)} = 50$$

Теоретически на наборе данных длиной 10000 пирамидальная сортировка должна работать в 376 раз быстрее сортировки слиянием, а на наборе 1000 в 50 раз быстрее.

сравнение практической сложности:

$$\frac{0.0405076}{0.0013914} = 31$$

$$\frac{0.0004761}{0.0001059} = 5$$

Практически на наборе данных длиной 10000 пирамидальная сортировка работает в 31 раз быстрее сортировки слиянием, а на наборе 1000 в 5 раз быстрее.

Разница между теоретическими и практическими результатами более чем в 10 раз обусловлена тем, что пирамидальная сортировка хоть и проходит гораздо меньше итераций, но на каждой итерации требует значительно больше действий, таких как сравнение, сложение, умножение и присваивание.

Но как мы видим встроенная функции сортировки языка c++ (sort) работает примерно с той же скоростью, что и функция пирамидальной. Это значит что быстрая сортировка на практике примерно в 10 раз медленнее, чем в теории.

По таблице результатов и расчётам также видна следующая тенденция: чем больше данных, тем значительнее разница в скорости работы алгоритмов пирамидальной сортировка и сортировки выбором.

Разница в скорости примерно равна $\frac{n}{20 * \log_2(n)}$ где n - количество данных в наборе.

Вывод

Если массив состоит более чем из 100 элементов то гораздо выгоднее по времени использовать пирамидальную сортировку. Выигрыш по скорости будет примерно равен

$$\frac{n}{20 * \log_2(n)}$$

ЗАКЛЮЧЕНИЕ

В ходе работы были проанализированы и протестированы скорость работы двух алгоритмов сортировки: пирамидальной сортировки и сортировки выбором. Итогом стало то, что ожидаемый результат полученный теоретически, примерно в 10 раз больше того, который получен опытным путём. Это связано с тем что при расчёте сложности учитывается только количество итераций циклов при работе алгоритма, но не операции происходящие непосредственно на каждой итерации. В данной работе это сыграло важную роль, так как техническая реализация пирамидальной сортировки сложнее и объёмнее (то есть на каждой итерации выполняется больше действий), чем сортировки выбором.

Тем не менее пирамидальной сортировка, которая является разновидностью быстрой сортировки (то есть сортировки, которая выполняется за $O(n * \log n)$), на средних (более 100 элементов) и больших (более 1000 элементов) наборах данных показала значительно лучший результат, а самое главное, что чем больше данных, тем больше выигрыш во времени. Он равен примерно $n / (20 * \log_2(n))$.

Таким образом использование пирамидальной сортировки предпочтительно по сравнению с сортировкой выбором для решения любых задач. Это подтверждается и тем что пирамидальная сортировка показала результат очень близкий к тому, который показала встроенная функция языка c++.

Сортировка выбором может использоваться для обучения, чтобы на простом примере дать представление о базовом алгоритме сортировки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Cppreference. `std::sort` [Электронный ресурс]. – URL: <https://en.cppreference.com/w/cpp/algorithm/sort.html> (дата обращения: 20.12.2025).
2. Habr. Пирамидальная сортировка (HeapSort) [Электронный ресурс]. – URL: <https://habr.com/ru/companies/otus/articles/460087> (дата обращения: 17.11.2025).
3. Microsoft. `<random>` [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/cpp/standard-library/random?view=msvc-170> (дата обращения: 16.11.2025)
4. Metanit. Чтение и запись текстовых файлов [Электронный ресурс]. – URL: <https://metanit.com/cpp/tutorial/8.3.php> (дата обращения: 16.11.2025)
5. Neerc.ifmo. Сортировка выбором [Электронный ресурс]. – URL: https://neerc.ifmo.ru/wiki/index.php?title=%D0%A1%D0%BE%D1%80%D1%82%D0%B8%D1%80%D0%BE%D0%B2%D0%BA%D0%B0_%D0%B2%D1%8B%D0%B1%D0%BE%D1%80%D0%BE%D0%BC (дата обращения: 17.11.2025).
6. Neerc.ifmo. Сортировка кучей [Электронный ресурс]. – URL: https://neerc.ifmo.ru/wiki/index.php?title=%D0%A1%D0%BE%D1%80%D1%82%D0%B8%D1%80%D0%BE%D0%B2%D0%BA%D0%B0_%D0%BA%D1%83%D1%87%D0%B5%D0%B9 (дата обращения: 18.11.2025).